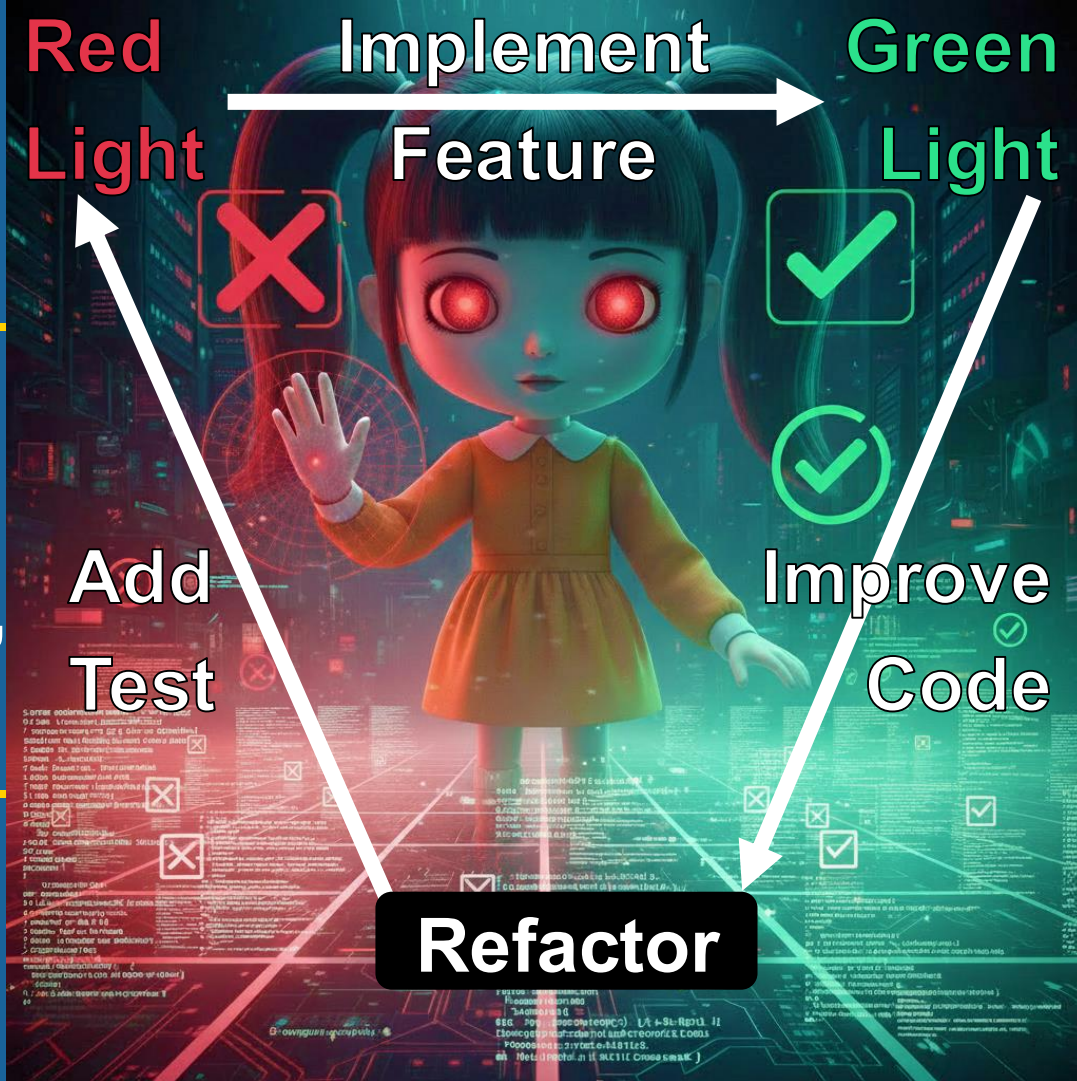


CS 35L Software Construction

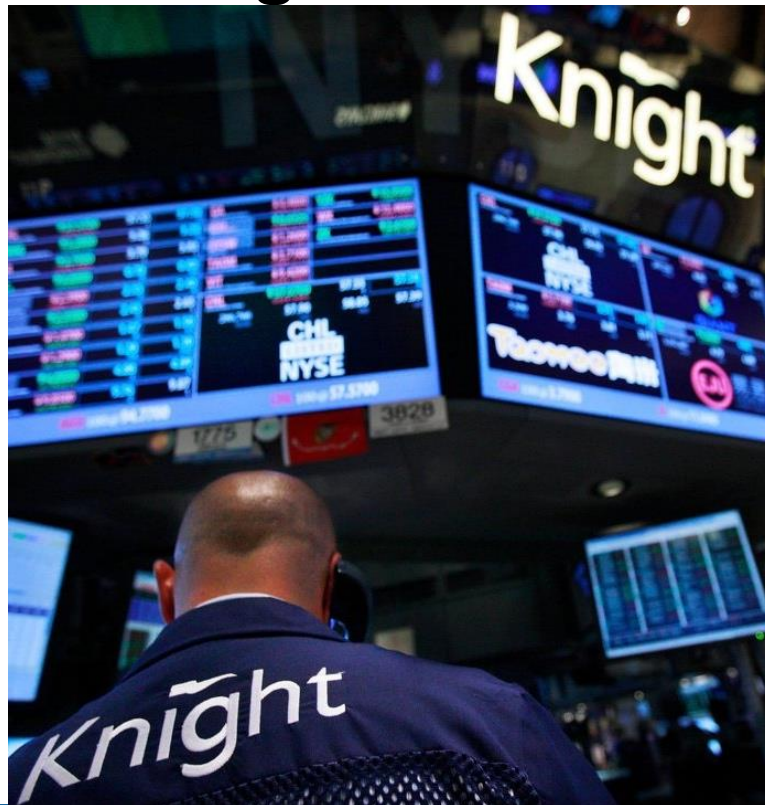
Lecture 9 – Testing, TDD & Refactoring

Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department



It's Software Engineering Storytime!

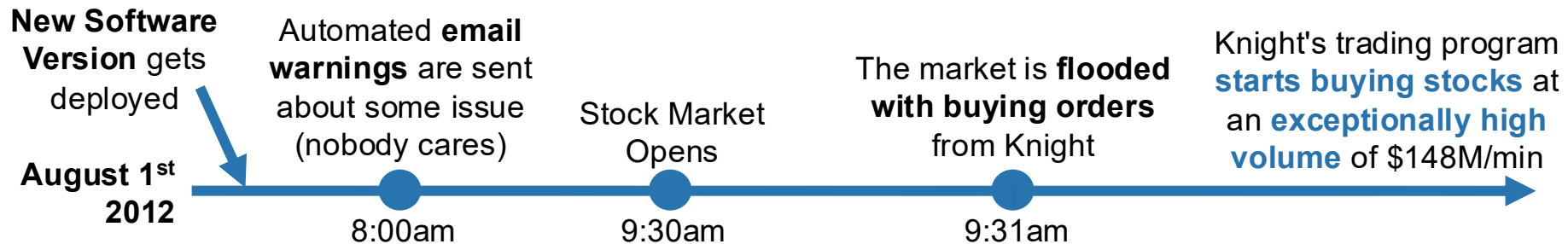
Once upon a Time, in the Kingdom of Wall Street There was a Knight...



Case Study: Knight Capital Group

A True Story

- **High-frequency Stock Trading**
 - **Buying** stocks at a **low** price and shortly after **selling** at a slightly **higher** price at **very high speeds**
- **17 years of dedicated work** made Knight one of the leading trading houses on Wall Street
- In 2012, Knight was the **largest trader** in U.S. stocks (**17% of daily trading volume**, \$21B/day)



Case Study: Knight Capital Group

A True Story

- **High-frequency Stock Trading**
 - **Buying** stocks at a **low** price and shortly after **selling** at a slightly **higher** price at **very high speeds**
- **17 years of dedicated work** made Knight one of the leading trading houses on Wall Street
- In 2012, Knight was the **largest trader** in U.S. stocks (**17% of daily trading volume, \$21B/day**)



New Software Version gets deployed

August 1st 2012

The market is **flooded** with **buying** orders from Knight

9:31am

Traders on Wall Street are puzzled why it keeeeeeeeps buying

9:32am

Knight's program is responsible for **50% of all trades** on the major stock exchanges

Knight rolls back the trading program to a previous version

Case Study: Knight Capital Group

A True Story

- **High-frequency Stock Trading**
 - **Buying** stocks at a **low** price and shortly after **selling** at a slightly **higher** price at **very high speeds**
- **17 years of dedicated work** made Knight one of the leading trading houses on Wall Street
- In 2012, Knight was the **largest trader** in U.S. stocks (**17% of daily trading volume, \$21B/day**)



New Software
Version gets
deployed

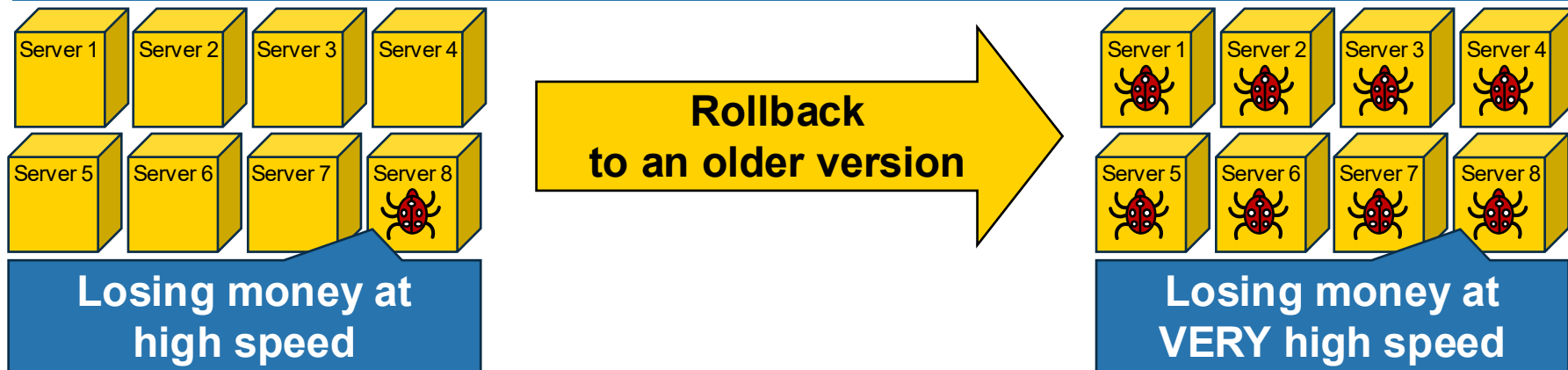
August 1st
2012

Knight rolls back the
trading program to a
previous version

Knight's trading program
buys 8x as many stocks

Case Study: Knight Capital Group

A True Story on losing \$440 Million in 28 Minutes



New Software
Version gets
deployed

August 1st
2012

Knight rolls back the
trading program to a
previous version

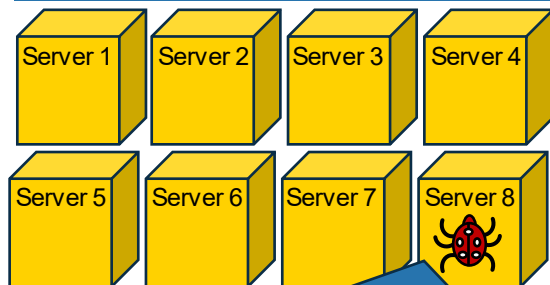
Knight's trading program
buys 8x as many stocks

Knight shuts down
everything

9:58am

Case Study: Knight Capital Group

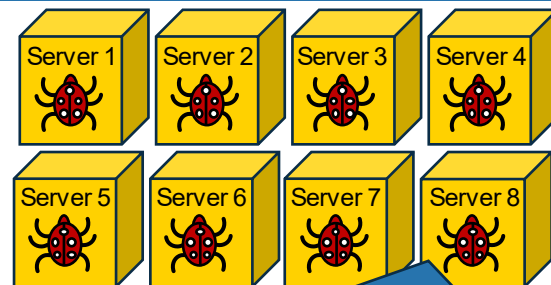
A True Story on losing \$440 Million in 28 Minutes



Losing money at
high speed



Has been dead code
for 9 years. “dead
code” = code that
will never be
executed



Losing money at
VERY high speed

```
//DO NOT USE IN PRODUCTION!!!!  
if (configurationFlag("PowerPeg")) {  
    buyHigh();  
    sellLow(); //loses money  
}
```



```
//some new code for  
//the new feature used the same name  
activateConfigurationFlag("PowerPeg");
```

Case Study: Knight Capital Group

A True Story on losing \$440 Million in 28 Minutes

Propose a generalizable Software Engineering Principle that would have prevented this!
Talk to your neighbor(s)!

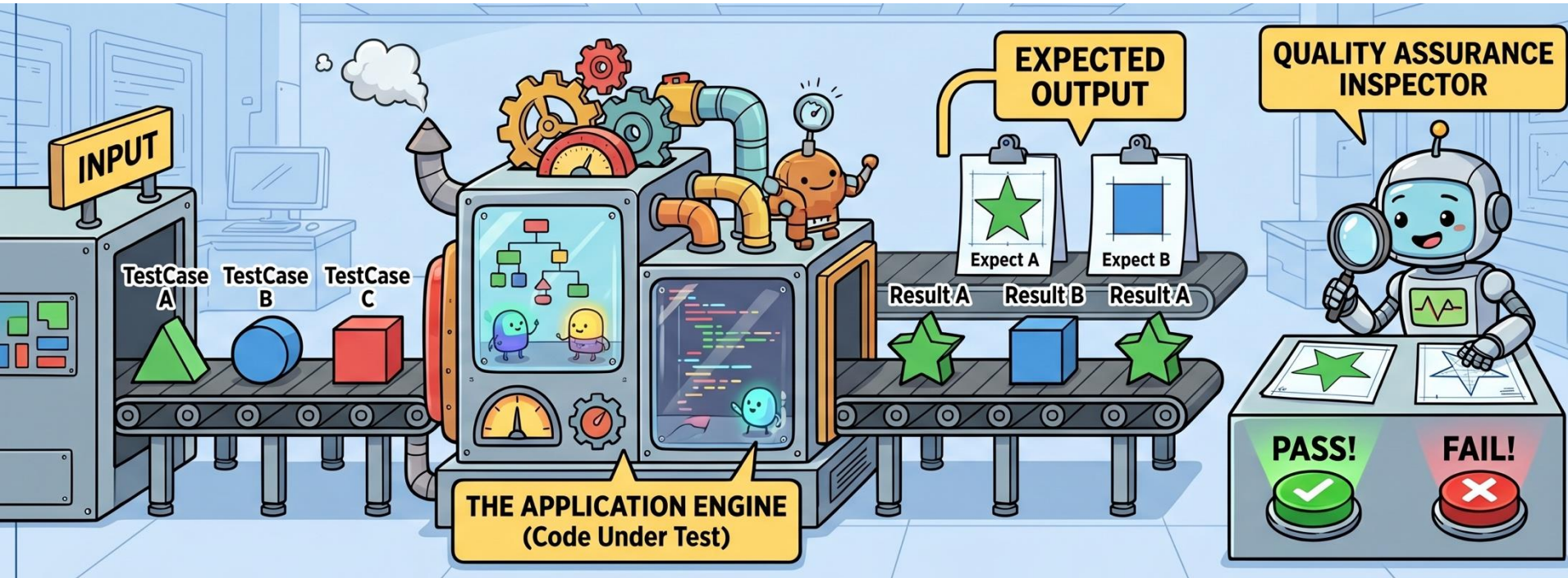
Has been dead code for 9 years. “dead code” = code that will never be executed

```
//DO NOT USE IN PRODUCTION!!!!  
if (configurationFlag("PowerPeg")) {  
    buyHigh();  
    sellLow(); //loses money  
}
```



```
//some new code for  
//the new feature used the same name  
activateConfigurationFlag("PowerPeg");
```


Lesson Learned: Always Test Software Before Deployment



Lesson Learned: Always Test Software Before Deployment

Definition of Software Testing

Executing the **program** with **selected inputs** in a **controlled environment** while checking **assertions on its behavior & outputs**

Goals of Software Testing

- **Primary: Reveal bugs**, so they can be fixed
 - **Testing can only prove the existence of bugs**
(not their absence!)
- **Secondary: Document** what the code **should** do

Lesson Learned: Always Test Software Before Deployment

Definition of Software Testing

Executing the program with selected inputs in a controlled environment while checking assertions on its behavior & outputs

my_domain_class.py

```
def calculate_price(price, discount_percent):  
    return price - discount_percent
```

```
def calculate_price(price, discount_percent):  
    return price - discount_percent * price/100
```

assert fails if the condition is false

my_test.py

```
def test_50_discount():  
    price = calculate_price(100, 50)  
    assert price == 50
```

```
def test_discount():  
    price = calculate_price(20, 50)  
    assert price == 10
```

Terminal

```
$pytest  
tests/test_main.py .F.....F...
```

Pytest is a common testing framework for Python

(calls all functions starting with "test" in .py files in the /tests folder of a Python project)

[100%]

```
===== 2 failed, 9 passed in 0.03s =====
```

Tests can be Automated in your Continuous Integration Workflow

- Continuous Integration (CI) is a software development practice in which changes are **frequently merged** into the main branch
- Tool support for this allows you to automatically run tests in the CI environment (on a server that has your code) to see if a change broke the build
- Convention: **Only push changes that should pass the tests**
- 31.5% of all NPM repositories on GitHub show badges for their build status [2]

Common CI Tools

- Jenkins
- Travis CI
- GitHub Actions



[2] A. Trockman et al. "Adding Sparkle to Social Coding: An Empirical Study of Repository Badges in the npm Ecosystem". ICSE 2018

Not All V&V (Verification & Validation) Can be Automated

Verification

Did we **build the system right**?



Can often be **automated**

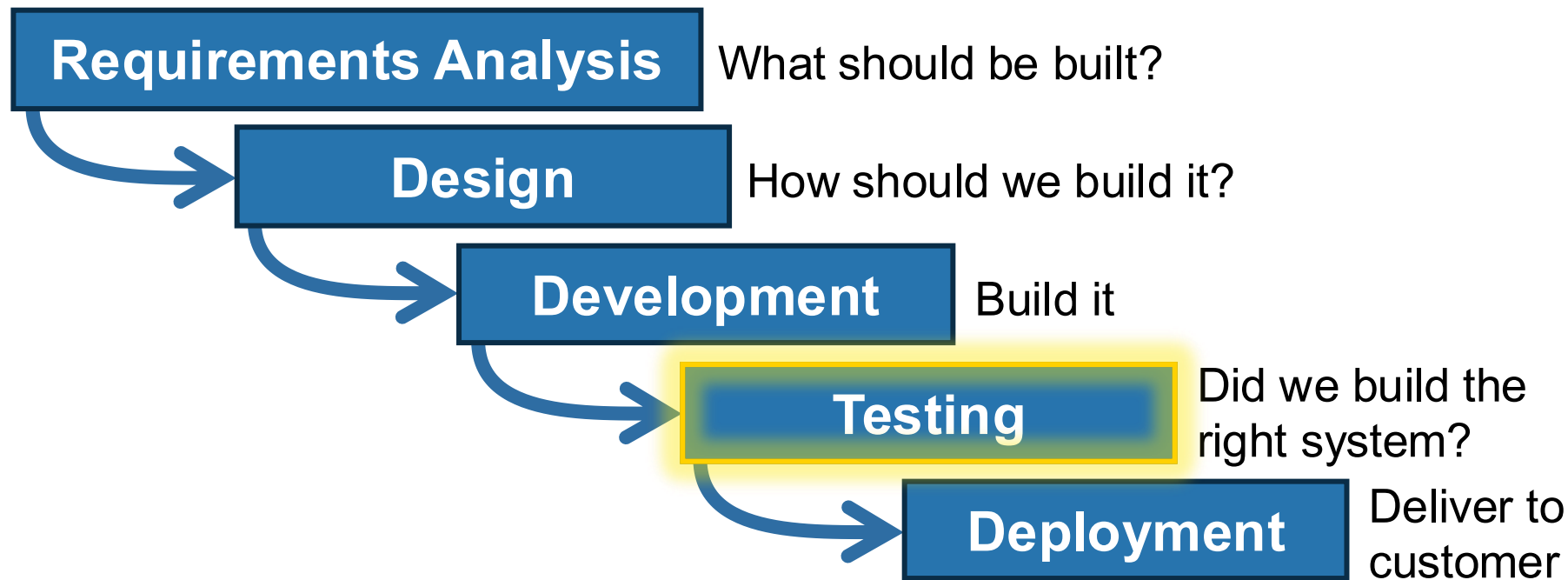
Validation

Did we **build the right system**?



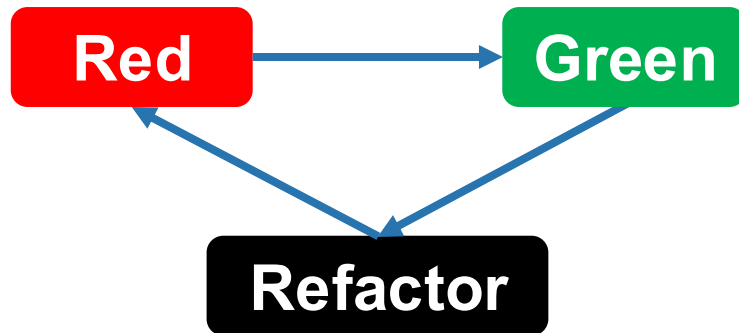
Must be done **manually**
(by someone who
represents real users)

Traditionally Testing Has Happened After Development



Test-Driven Development (TDD) Puts Testing First

Originally proposed by **Kent Beck**
(who later went on to work for Facebook/Meta)



Red

For your new requirement write a **small test that fails**, and perhaps doesn't even compile at first

Green

Make the **test pass** with **minimal coding effort**, potentially using simplifying shortcuts in the process

Refactor

Make the design more **elegant**, cleaner, and potentially **faster** while **not changing the functionality** of the program

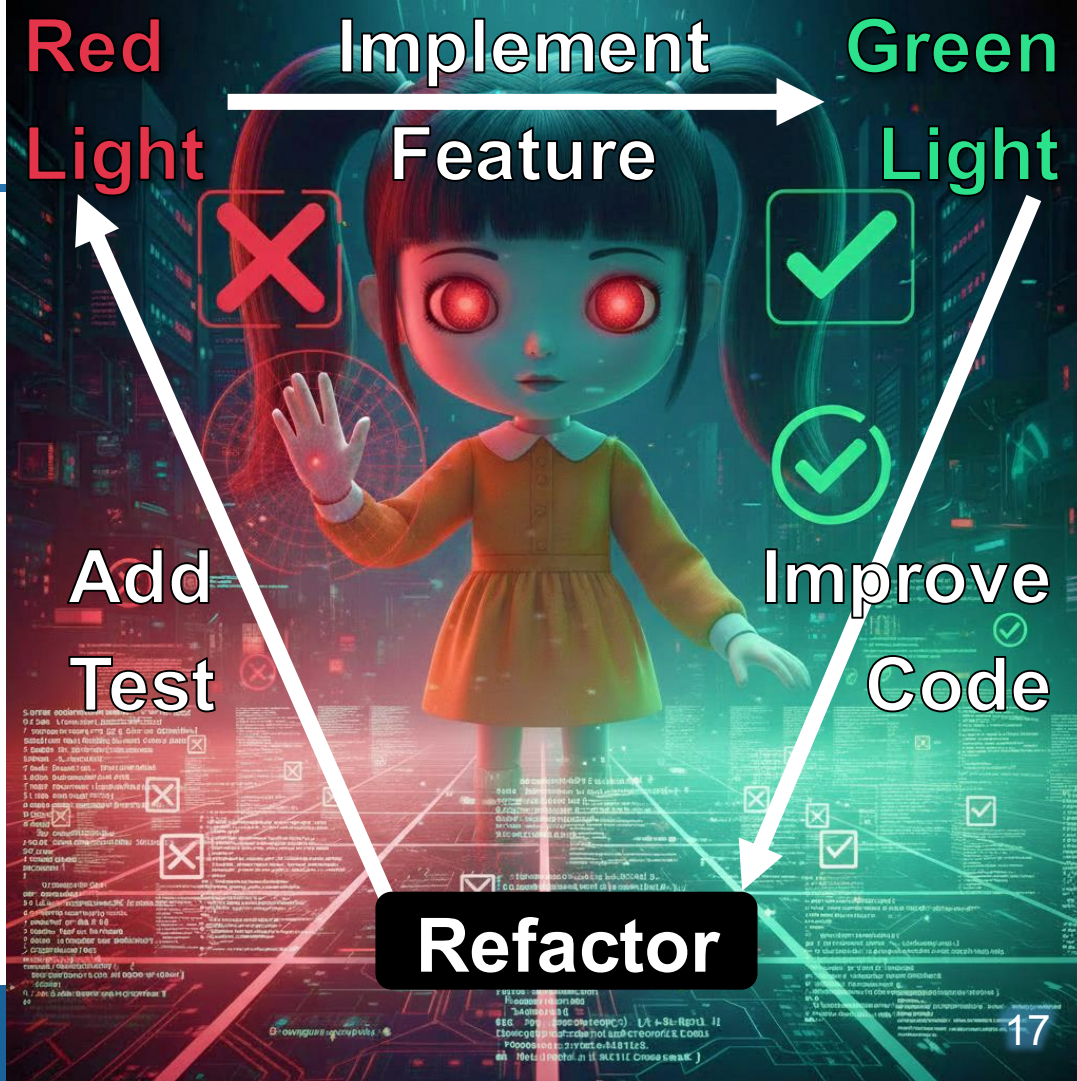
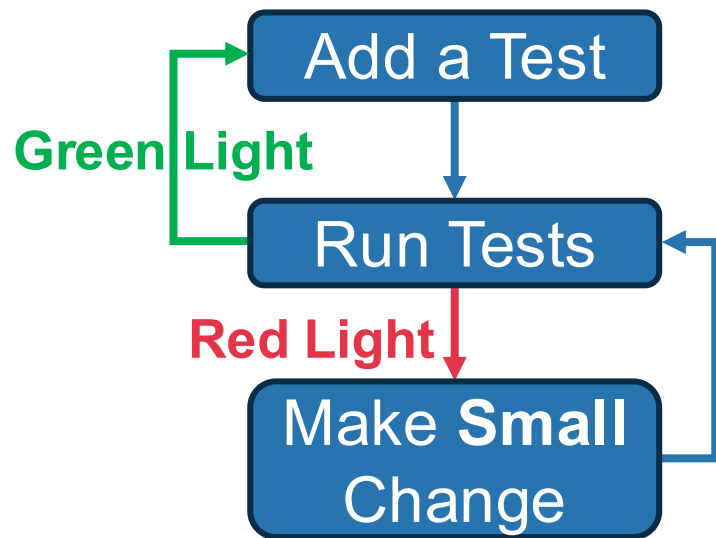
More on this in "[*Test Driven Development: By Example*](#)" by Kent Beck

Refactoring Turns **Code Smells** into **Clean Code**

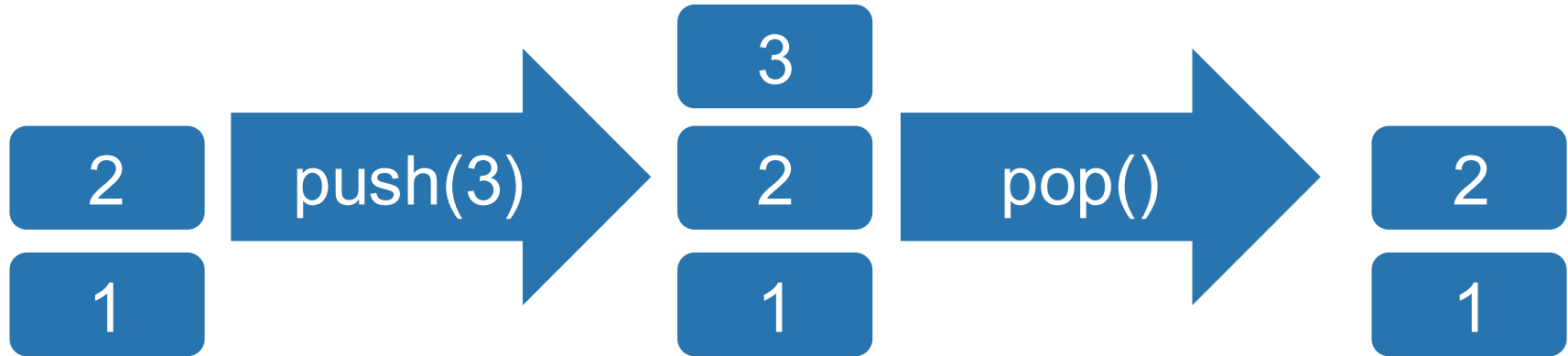
- Motivation: “When a software system evolves, its **complexity increases** unless work is done to reduce it” (Second Law of Software Evolution [1])
- Refactoring **reduces** accidental **complexity** in software systems and/or makes them easier to read / understand
- Code is read more often than it is written, so **make sure your code is readable**
- Common themes in refactoring:
 - Avoid **code duplication** by creating **abstractions**
 - Remove **dead code** & improve **documentation**
 - Replace **conditional logic** with **polymorphism**
 - **Improve naming** of classes, functions, & variables
 - Learn common refactoring techniques here: <https://refactoring.guru/refactoring/catalog>

[1] M. M. Lehman et al. "Metrics and laws of software evolution-the nineties view," *Proc. International Software Metrics Symposium* 1997

TDD Mantra



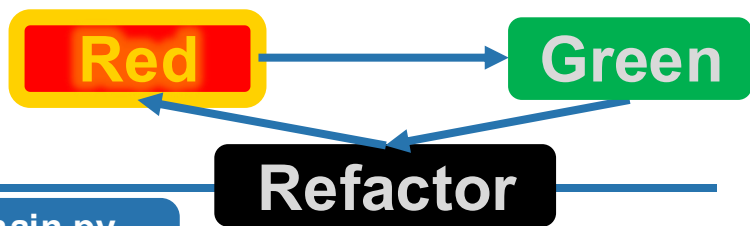
TDD Example: A Simple Stack Implementation



A stack is a list data structure that operates on the Last-In, First-Out (LIFO) principle.

Walk through this at home: https://github.com/UCLA-CS-35L/TDD_stack_example

Start by Writing a Failing Test



my_stack.py

test_main.py

```
from src.my_stack.my_stack import MyStack

def test_create_stack():
    stack = MyStack()
```

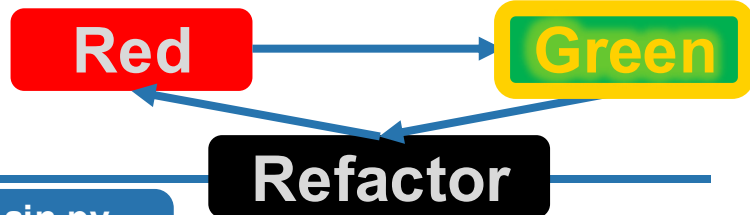
We want to see the test fail first! This step is “testing the test”. Otherwise, we can’t be sure that the test works. (maybe we accidentally referenced the stack class from a library?)

Terminal

```
$pytest
tests/test_main.py:1: in <module>
    from src.my_stack.my_stack import MyStack
E   ImportError: cannot import name 'MyStack' from 'src.my_stack.my_stack'
Did you mean: 'my_stack'?
```

```
===== short test summary info =====
ERROR tests/test_main.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.04s =====
```


Then Add the Simplest Code that Passes the Test



my_stack.py

```
class MyStack:  
    pass
```

Null operation in Python (to tell the interpreter the class is empty)

test_main.py

```
from src.my_stack.my_stack import MyStack  
  
def test_create_stack():  
    stack = MyStack()
```

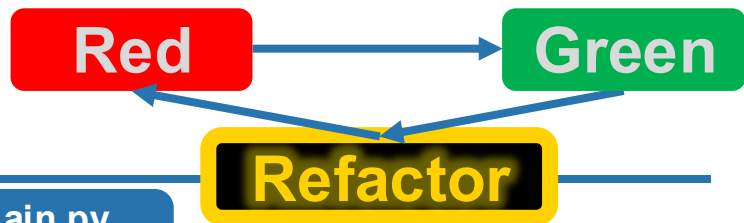
If we had written more code than necessary to pass the test, we **would not know whether the additional code is correct**, since we don't have a test for it.

Terminal

```
$pytest
```

```
===== test session starts =====  
tests/test_main.py . [100%]  
  
===== 1 passed in 0.00s =====
```

Improve the Quality of your Code via Refactoring



my_stack.py

```
class MyStack:
    """
    A simple implementation of a Stack data
    structure.
    A Stack operates on the Last-In, First-
    Out (LIFO) principle.
    """
    pass
```

test_main.py

```
from src.my_stack.my_stack import MyStack

def test_create_stack():
    stack = MyStack()
```

Comments should **describe only a fixed contract** of what this class does across future versions (and ideally also why)

Tests give us the **confidence** that our **refactoring** didn't **break anything**. Everything still works!

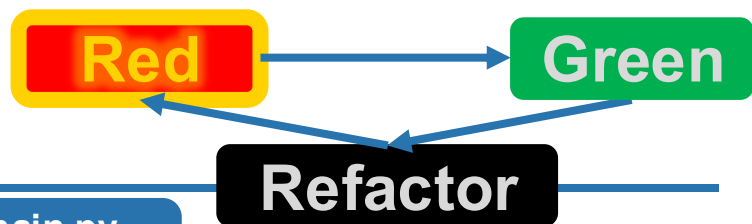
Terminal

\$pytest

```
===== test session starts =====
tests/test_main.py . [100%]

===== 1 passed in 0.00s =====
```

Add Another Test for New Functionality



my_stack.py

```
class MyStack:
    """
    A simple implementation of a Stack data
    structure.
    A Stack operates on the Last-In, First-
    Out (LIFO) principle.
    """
    pass
```

test_main.py

```
from src.my_stack.my_stack import MyStack

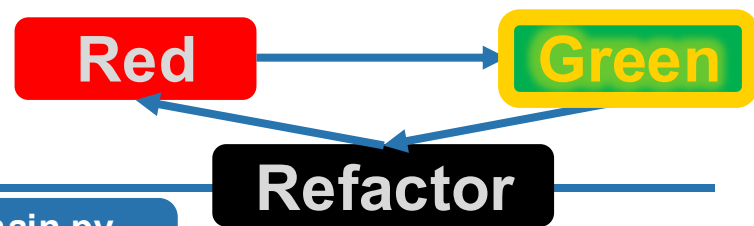
def test_create_stack():
    stack = MyStack()
    assert stack.is_empty()
```

Writing the **code** using the method before its implementation leads to more **modular design**

Terminal

```
def test_create_stack():
    stack = MyStack()
>    assert stack.is_empty()
           ^^^^^^^^^^^^^^^^^
E      AttributeError: 'MyStack' object has no attribute 'is_empty'
===== 1 failed in 0.02s =====
```

Then Add the Simplest Code that Passes the Tests



my_stack.py

```
class MyStack:
    """
    A simple implementation of a Stack data
    structure.
    A Stack operates on the Last-In, First-
    Out (LIFO) principle.
    """
```

```
def is_empty(self):
    return True
```

This is the simplest implementation to pass the test.
We have not tested other non-empty stacks yet!
Hence, we will only implement it when we have a test for this.

test_main.py

```
from src.my_stack.my_stack import MyStack

def test_create_stack():
    stack = MyStack()
    assert stack.is_empty()
```

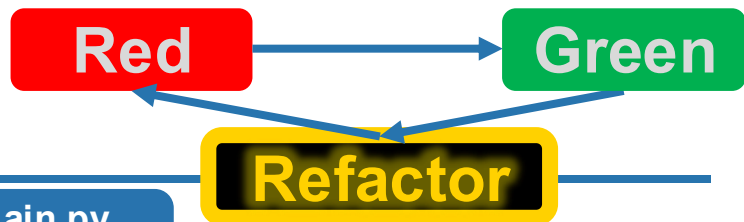
Terminal

\$pytest

```
===== test session starts =====
tests/test_main.py . [100%]

===== 1 passed in 0.00s =====
```

Improve the Quality of your Code via Refactoring



my_stack.py

```
class MyStack:
    def is_empty(self):
        """
        Checks whether the stack is currently
        empty.
        Returns:
            bool: True if the stack contains no
            elements, False otherwise.
        """
        return True
```

test_main.py

```
from src.my_stack.my_stack import MyStack

def test_create_stack():
    stack = MyStack()
    assert stack.is_empty()
```

Our code is still very clean, the only thing we can do is **documenting** the contract of this method

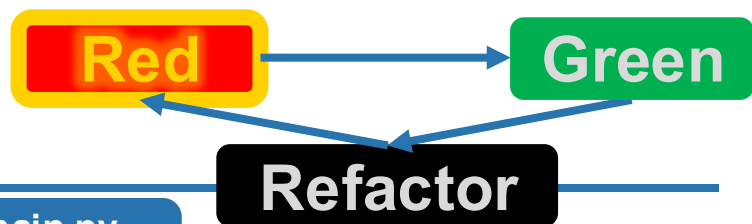
Terminal

\$pytest

```
===== test session starts =====
tests/test_main.py . [100%]

===== 1 passed in 0.00s =====
```

Add Another Test for New Functionality



my_stack.py

```
class MyStack:
    def is_empty(self):
        """ ... """
        return True
```

We try to write simple, small tests, ideally the simplest test that fails

test_main.py

```
from src.my_stack.my_stack import MyStack

def test_create_stack():
    stack = MyStack()
    assert stack.is_empty()

def test_add_element():
    stack = MyStack()
    stack.add_element(3)
    assert not stack.is_empty()
```

Terminal

```
$pytest
tests/test_main.py .F
```

[100%]

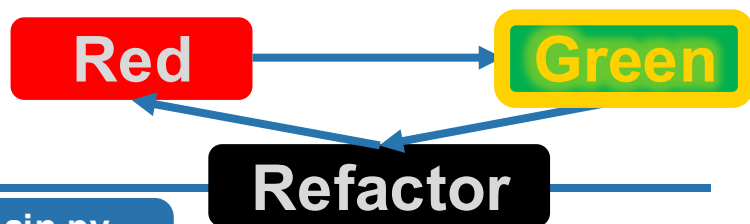
```
_____ test_add_element _____
```

```
def test_add_element():
    stack = MyStack()
>    stack.add_element(3)
    ^^^^^^^^^^^^^^^^^^^
```

```
E      AttributeError: 'MyStack' object has no attribute 'add_element'
```

```
===== 1 failed, 1 passed in 0.02s =====
```


Now we Actually Need to Think and Add Lots of Code



my_stack.py

```
class MyStack:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        """ ... """
        return len(self.elements) == 0

    def add_element(self, element):
        self.elements.append(element)
```

test_main.py

```
from src.my_stack.my_stack import MyStack

def test_create_stack():
    stack = MyStack()
    assert stack.is_empty()

def test_add_element():
    stack = MyStack()
    stack.add_element(3)
    assert not stack.is_empty()
```

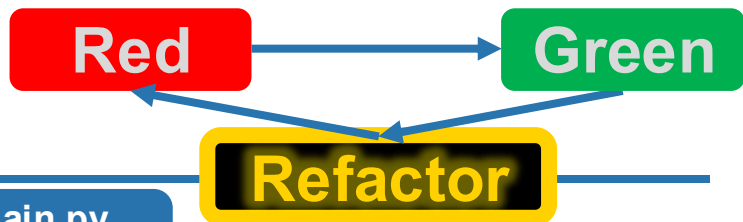
Terminal

\$pytest

```
===== test session starts =====
tests/test_main.py .. [100%]

===== 2 passed in 0.00s =====
```

Choose Better Method Names



my_stack.py

```
class MyStack:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        """ ... """
        return len(self.elements) == 0

    def push(self, element):
        self.elements.append(element)
```

test_main.py

```
from src.my_stack.my_stack import MyStack

def test_create_stack():
    stack = MyStack()
    assert stack.is_empty()

def test_push():
    stack = MyStack()
    stack.push(3)
    assert not stack.is_empty()
```

Terminal

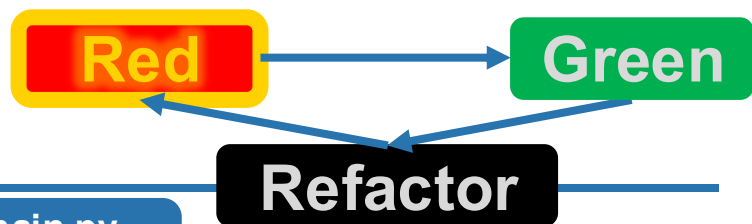
\$pytest

Thanks to our tests we know this refactoring didn't break anything

```
===== test session starts =====
tests/test_main.py .. [100%]

===== 2 passed in 0.00s =====
```

Add Another Test for New Functionality



my_stack.py

```
class MyStack:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        """ ... """
        return len(self.elements) == 0

    def push(self, element):
        self.elements.append(element)
```

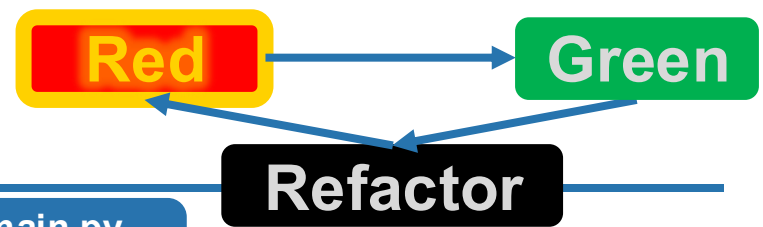
test_main.py

```
from src.my_stack.my_stack import MyStack
...
def test_pop():
    stack = MyStack()
    stack.push(3)
    element = stack.pop()
    assert element == 3
    assert stack.is_empty()
```

Terminal

```
$pytest
tests/test_main.py .F [100%]
_____ test_add_element _____
def test_pop():
>     element = stack.pop()
                        ^^^^^^
E         AttributeError: 'MyStack' object has no attribute 'pop'
===== 1 failed, 2 passed in 0.02s =====
```

Then Add the Simplest Code that Passes the Tests



my_stack.py

```
class MyStack:
    def __init__(self):
        self.elements = []
    ...
    def push(self, element):
        self.elements.append(element)

    def pop(self):
        element = self.elements[-1]
        return element
```

test_main.py

```
from src.my_stack.my_stack import MyStack
...
def test_pop():
    stack = MyStack()
    stack.push(3)
    element = stack.pop()
    assert element == 3
    assert stack.is_empty()
```

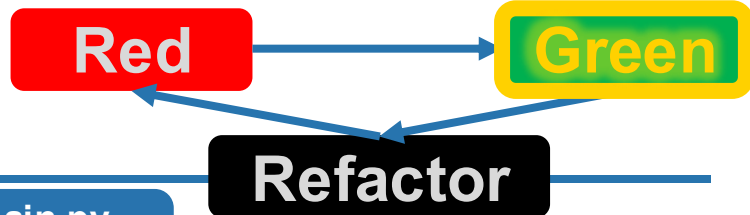
Terminal

```
$pytest
tests/test_main.py .F
```

Thanks to our tests we know that we made a mistake in the code. We didn't change the size of the stack!

```
_____ test_add_element _____ [100%]
> assert stack.is_empty()
E      assert False
E          + where False = is_empty()
E          +     where is_empty = <src.my_stack.my_stack.MyStack object at 0x10aa30690>.is_empty
===== 1 failed, 2 passed in 0.02s =====
```

Fix the Code Until All Tests Pass



my_stack.py

```
class MyStack:
    def __init__(self):
        self.elements = []
    ...
    def push(self, element):
        self.elements.append(element)

    def pop(self):
        element = self.elements[-1]
        self.elements = self.elements[0:-1]
        return element
```

test_main.py

```
from src.my_stack.my_stack import MyStack
...
def test_pop():
    stack = MyStack()
    stack.push(3)
    element = stack.pop()
    assert element == 3
    assert stack.is_empty()
```

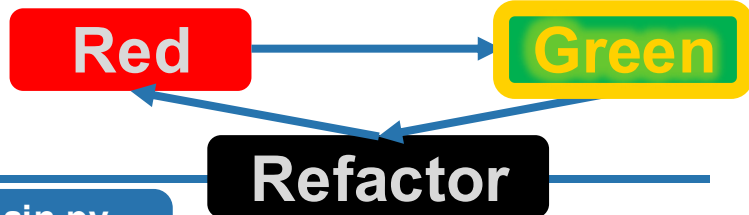
Now we have confidence that our stack pops correctly for one element

Terminal

\$pytest

```
===== test session starts =====
tests/test_main.py .. [100%]
===== 3 passed in 0.02s =====
```

Does the Stack work correctly with Multiple Elements too?



my_stack.py

```
class MyStack:
    def __init__(self):
        self.elements = []
    ...
    def push(self, element):
        self.elements.append(element)

    def pop(self):
        element = self.elements[-1]
        self.elements = self.elements[0:-1]
        return element
```

test_main.py

```
def test_multiple_pop():
    stack = MyStack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    element = stack.pop()
    assert element == 3
    element = stack.pop()
    assert element == 2
    element = stack.pop()
    assert element == 1
    assert stack.is_empty()
```

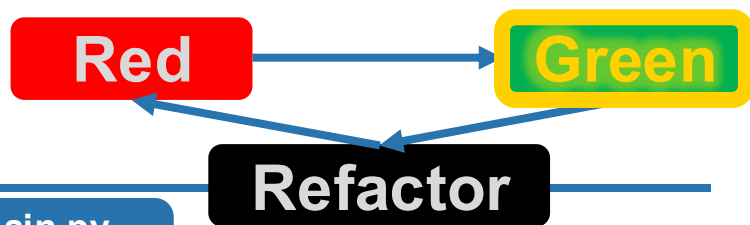
Terminal

Already passes Yay!

\$pytest

```
===== test session starts =====
tests/test_main.py .. [100%]
===== 4 passed in 0.02s =====
```

Is there any Redundant Code in the Tests?



my_stack.py

```
class MyStack:
    def __init__(self):
        self.elements = []
    ...
    def push(self, element):
        self.elements.append(element)

    def pop(self):
        element = self.elements[-1]
        self.elements = self.elements[0:-1]
        return element
```

test_main.py

```
def test_create_stack():
    stack = MyStack()
    assert stack.is_empty()

def test_push():
    stack = MyStack()
    stack.push(3)
    assert not stack.is_empty()

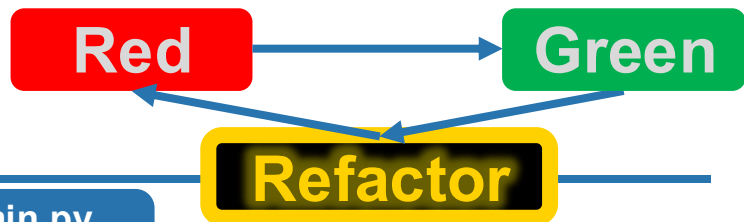
def test_multiple_pop():
    stack = MyStack()
    stack.push(1)
    ...
```

Terminal

\$pytest

```
===== test session starts =====
tests/test_main.py .. [100%]
===== 4 passed in 0.02s =====
```


Fixtures are Objects Created Freshly for each Test Case



Fixtures are especially useful for **complex setup / teardown procedures**, such as setting up a test database

```
@pytest.fixture
def db_connection():
    # 1. Setup: Connect to the test database
    db = db_library.connect(TEST_DB_URL)
    db.add_data(TEST_DATA)

    # 2. Yield: Forward connection object to test
    yield conn

    # 3. Teardown: Close the connection
    db.rollback()
    db.close()
```

Read more here: <https://docs.pytest.org/en/stable/how-to/fixtures.html>

test_main.py

```
@pytest.fixture
def stack():
    # Defines stack fixture

    # Setup Phase: Code run before each test
    stack = MyStack()
    yield stack
    # Runs the test with fixture

    # Teardown Phase: Code run after each test

def test_create_stack(stack):
    assert stack.is_empty()
    # Uses stack fixture

def test_push(stack):
    stack.push(3)
    # Uses stack fixture
    assert not stack.is_empty()
```

\$pytest

```
===== test session starts =====
tests/test_main.py .. [100%]
===== 4 passed in 0.02s =====
```

TDD Is Not Just About Finding Bugs

Tests are living documentation

Tests describe what the code **should do** and are always **up-to-date**.

TDD often leads to better design

Writing the test first makes you think about **how to use a new method / class first**, which often leads to a **more modular design**.

TDD can increase productivity

TDD mantra keeps you **focused** on the current task.

Read more in [I. Karac & B. Tunhan "What Do We \(Really\) Know about Test-Driven Development?" IEEE Software 2018](#)

Rules of Test-Driven Development (According to “Uncle Bob” Robert C. Martin)

- 1) **Do not write a line of production code until you have a failing unit test**
- 2) **Do not write more of a unit test than is sufficient for the test to fail**
(You cannot write a lot of unit test code before you switch back to writing production code)
- 3) **Do not write more production code than is sufficient to pass the tests**
⇒ You will end up writing unit tests & production code **somewhat concurrently**

This is NOT a dogma.

Every rule in software engineering has its exceptions.

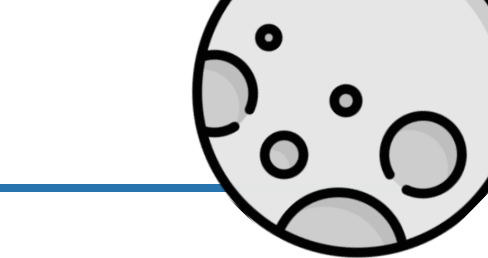
Once you have mastered TDD, you can adjust it to fit your coding style & project.

However, it's hard to master TDD without **first practicing it based on these rules.**

Limitations of TDD

TDD does not work well for:

- extremely complex behavior where the **solution to your problem is not an incremental improvement to simpler problems** (e.g. implementing ACID-compliant DBMS, distributed databases,)
- systems with **non-binary success outcomes** (e.g., image recognition, image processing)
- testing **non-functional properties** (e.g., response time, availability, ...)



**You'll never
get to the moon simply by
building taller and taller towers.**

Unit Testing Is not Enough

- Components that **work well in isolation** might **break once they are composed together**
- Integration tests test the interactions between two components

```
//DO NOT USE IN PRODUCTION!!!!  
if (configurationFlag("PowerPeg")) {  
    buyHigh();  
    sellLow(); //loses money  
}
```



Works fine in isolation but **not together with the other code**

```
//some new code for  
//the new feature used the same name  
activateConfigurationFlag("PowerPeg");
```



Levels of Software Tests

Tests whether the **complete application** from user input to output works correctly.

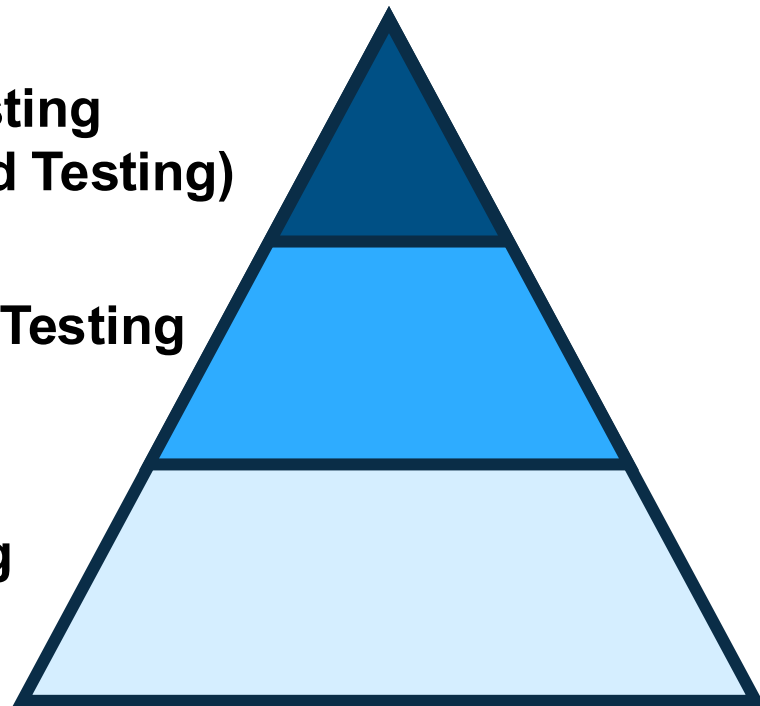
**System Testing
(End-to-End Testing)**

Tests whether different components, modules, or services **work together correctly**.

Integration Testing

Tests **individually testable units** of an application (e.g., functions, classes, components) **in isolation**.

Unit Testing



Cucumber Lets you Write Executable System Tests in Gherkin (User Story Acceptance Criteria)

Feature: Withdrawing cash

Rule: Customers cannot withdraw more than their balance

Scenario: Successful withdrawal within balance

Given Alice has 234.56 in their account

When Alice tries to withdraw 200.00

Then the withdrawal is successful

Scenario: Declined withdrawal in excess of balance

Given Hamza has 198.76 in their account

When Hamza tries to withdraw 200.00

Then the withdrawal is declined

Can be
automatically
translated into
executable
tests using
natural
language
processing

See <https://cucumber.io/>

End-to-end Web Testing Frameworks

```
await page.goto('https://localhost:8080/');
```

Page Navigation

```
await page.getByRole('textbox').fill('example value');
```

Entering Text

```
const submitLink = page.getByRole('link', { name: 'Submit' });  
await submitLink.click();
```

Clicking a Link

See more details here: <https://playwright.dev/docs/writing-tests>

End-to-end Web Testing Frameworks

```
await expect(page.getByText('Welcome')).toBeVisible();
```

```
// At least one of the two elements  
is visible, possibly both.
```

```
await expect(  
page.getByRole('button', { name: 'Sign in' })  
.or(page.getByRole('button', { name: 'Sign up' })))  
.first()  
) .toBeVisible();
```

Assertion for
Visibility

```
const locator = page.locator('.title');  
await expect(locator).toContainText('substring');  
await expect(locator).toContainText(/\d messages/);
```

Assertion for Text
Content

See more details here: <https://playwright.dev/docs/writing-tests>

How to Write Good Tests?

- Select Inputs that test the **boundaries & equivalence** classes (e.g., positive numbers, negative numbers, zero, very large, very small)
- Try to cover common edge cases, not just the "happy path"
- Write **strong assertions**! Your assertion should test ALL aspects of the specification (remember the acceptance criteria from your user stories? If you've written good ones, this part is easier)
- **Test behavior, not implementation details!**
 - Your test should pass any correct implementation. Write it as a specification of the behavior, not the design decisions details, because those might change!

Learn this based on concrete Python examples:

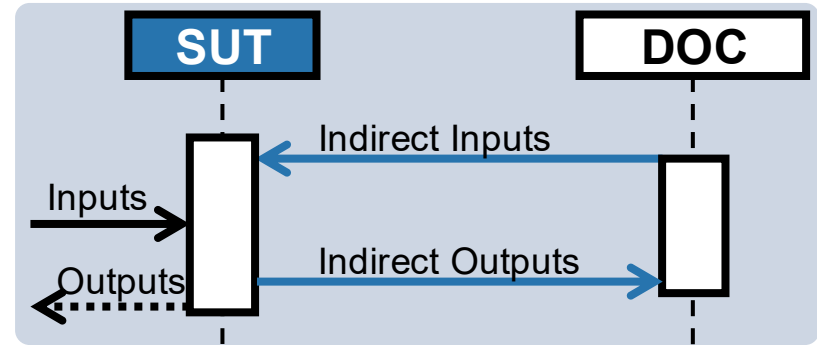
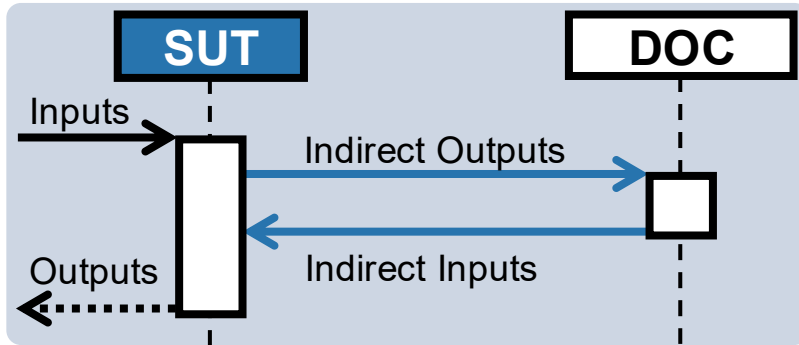
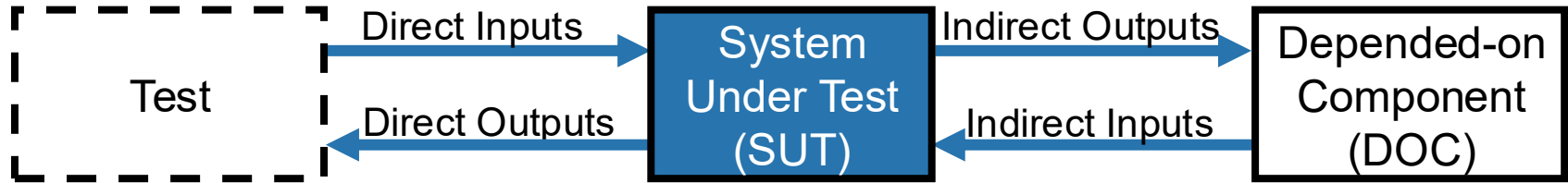
<https://tobiasduerschmid.github.io/SEBook/tools/testing-foundations-tutorial>

Indirect Inputs & Indirect Outputs

Can be Ordered in Many Different Sequences

Indirect inputs are values returned by another component whose services the SUT uses

Indirect outputs are actions that cannot be observed through the public API of the SUT which are seen or experienced by other systems or application components

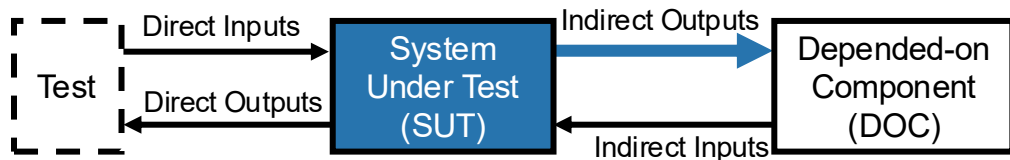


Mock Object Pattern

Problem: How to observe indirect outputs sent to separate DOCs?

Context: The connector between SUT and DOC cannot easily be intercepted.

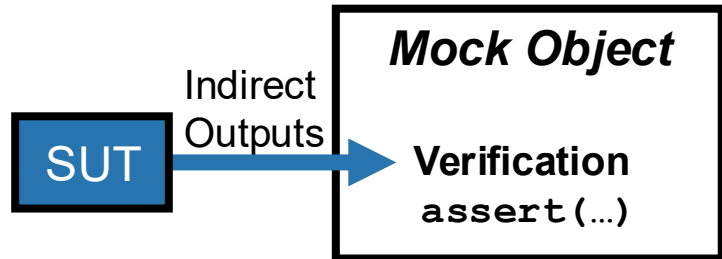
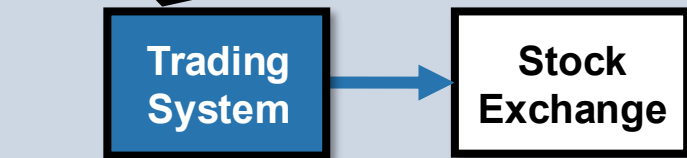
Solution: Create a **Mock Object** that replaces the **DOC** and only **verifies the indirect outputs**



More Details here: <http://xunitpatterns.com/Mock%20Object.html>

Different example: Testing that your app sends the right requests to an LLM-based AI provider API

How to test that the trading system **sends well-formed buy / sell orders** to the stock exchange without sending them to the actual stock exchange?

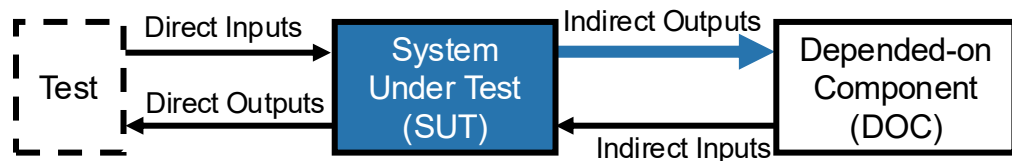


Test Spy Pattern

Problem: How to observe indirect outputs sent to separate DOCs?

Context: The connector between SUT and DOC cannot easily be intercepted.

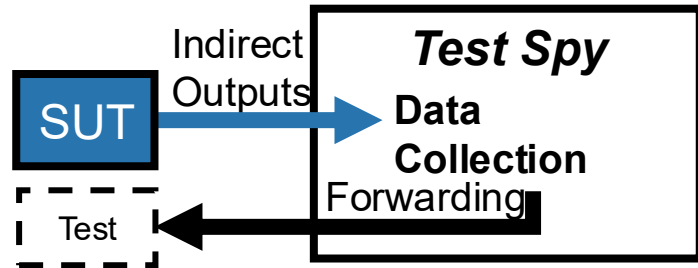
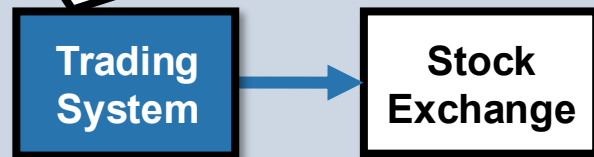
Solution: Create a **Test Spy** component that **replaces** the **DOC** and forwards the indirect outputs to the test.



More Details here: <http://xunitpatterns.com/Test%20Spy.html>

Different example: Testing that your system sends the requests in the correct order to an LLM provider via their API

How to test that the trading system sends buy / sell orders to the stock exchange in the **correct order**?



Test Stub Pattern

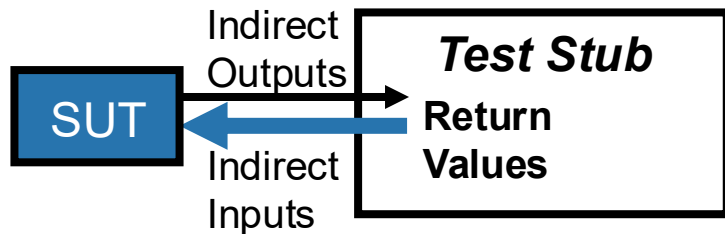
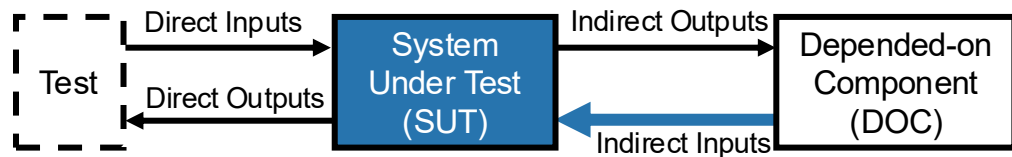
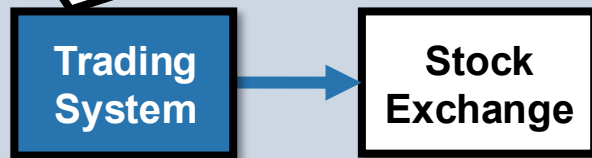
Different example: Testing that your system behaves correctly when receiving various responses from the LLM provider API

Problem: How to control indirect inputs sent from separate DOCs?

Context: The connector between SUT and DOC cannot easily be intercepted.

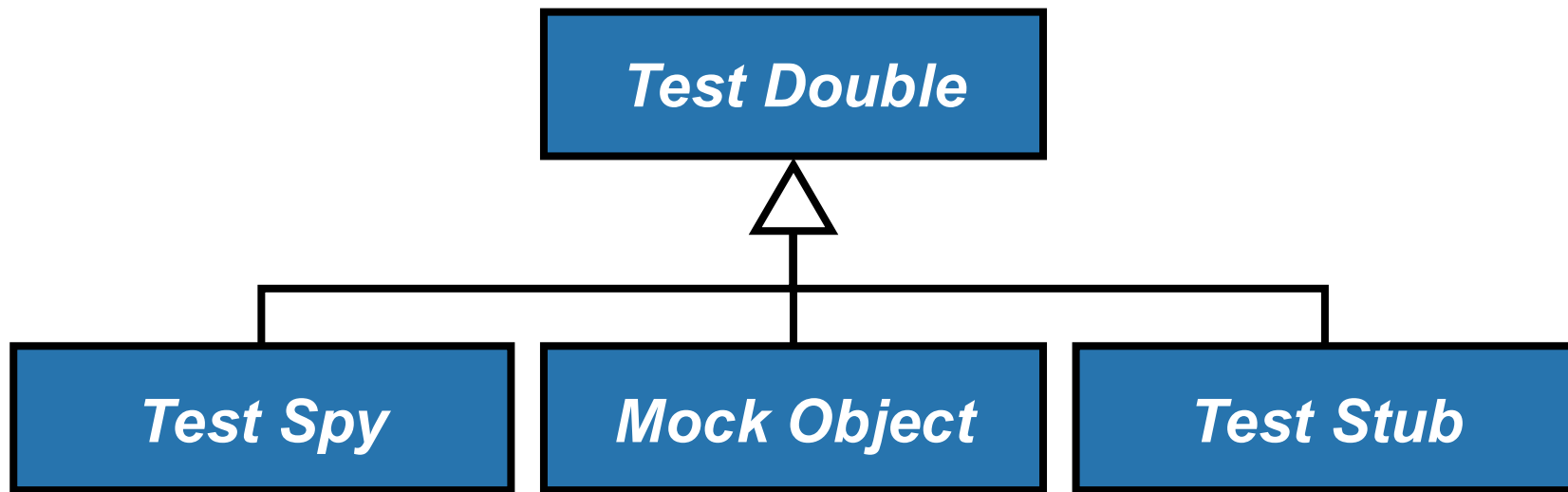
Solution: Create a **Test Stub** component that **replaces** the **DOC** and sends the desired inputs to the SUT.

How to test that the trading system responds correctly to updates on stock prices in the stock exchange?



More Details here: <http://xunitpatterns.com/Test%20Stub.html>

Test Spies, Mock Components, and Test Stubs are all unified under the term *Test Doubles*

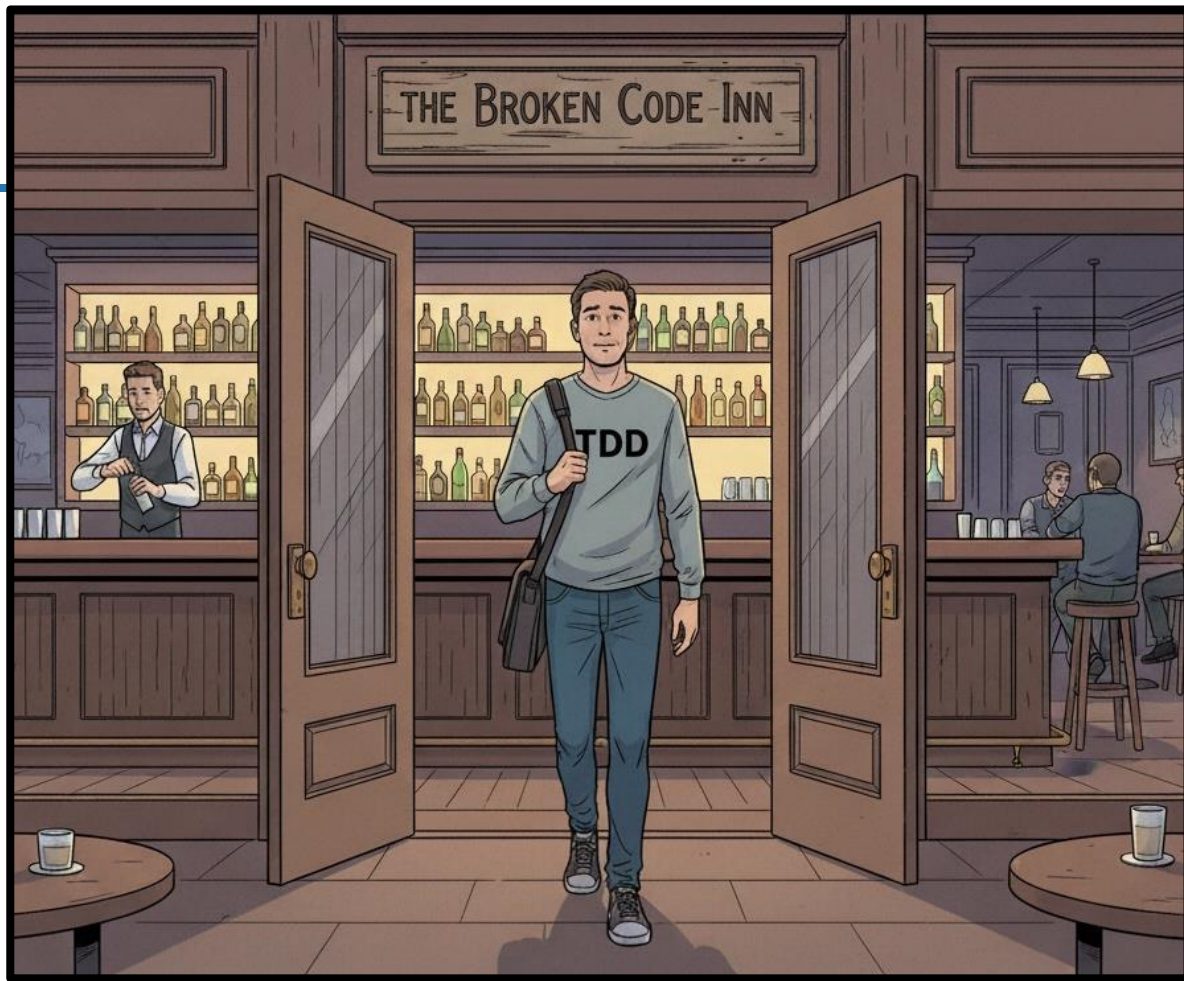


In Node.js HTTP test doubles can be implemented using this library:

<https://www.npmjs.com/package/node-mocks-http>

A Software Tester WALKS into a bar

The System
Correctly
Handles a
Standard
Request



A Software Tester RUNS into a bar

The System
Correctly
Handles a
Non-Standard
Request



A Software Tester CRAWLS into a bar

The System
Correctly
Handles a
Non-Standard
Request



A Software Tester DANCES into a bar

The System
Correctly
Handles a
Non-Standard
Request



A Software Tester FLIES into a bar

The System
Correctly
Handles a
Non-Standard
Request



A Software Tester Orders ...

The System
Correctly
Handles a
Standard
Request



A Software Tester Orders ...

The System
Correctly
Handles a
Standard
Request



A Software Tester Orders ...

The System
Correctly
Handles a
Non-Standard
Request



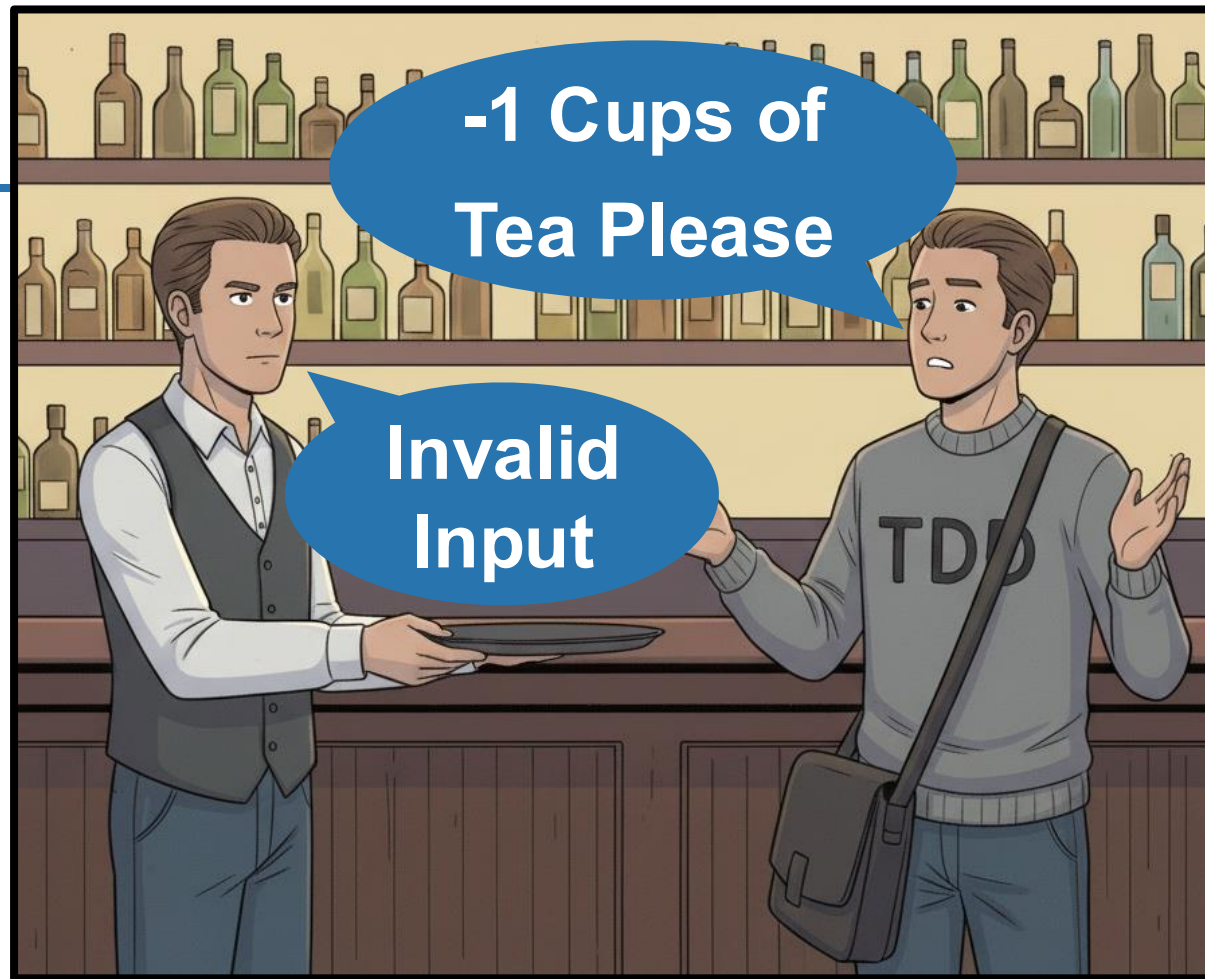
A Software Tester Orders ...

The System
Correctly
Handles a
Very Large
Request



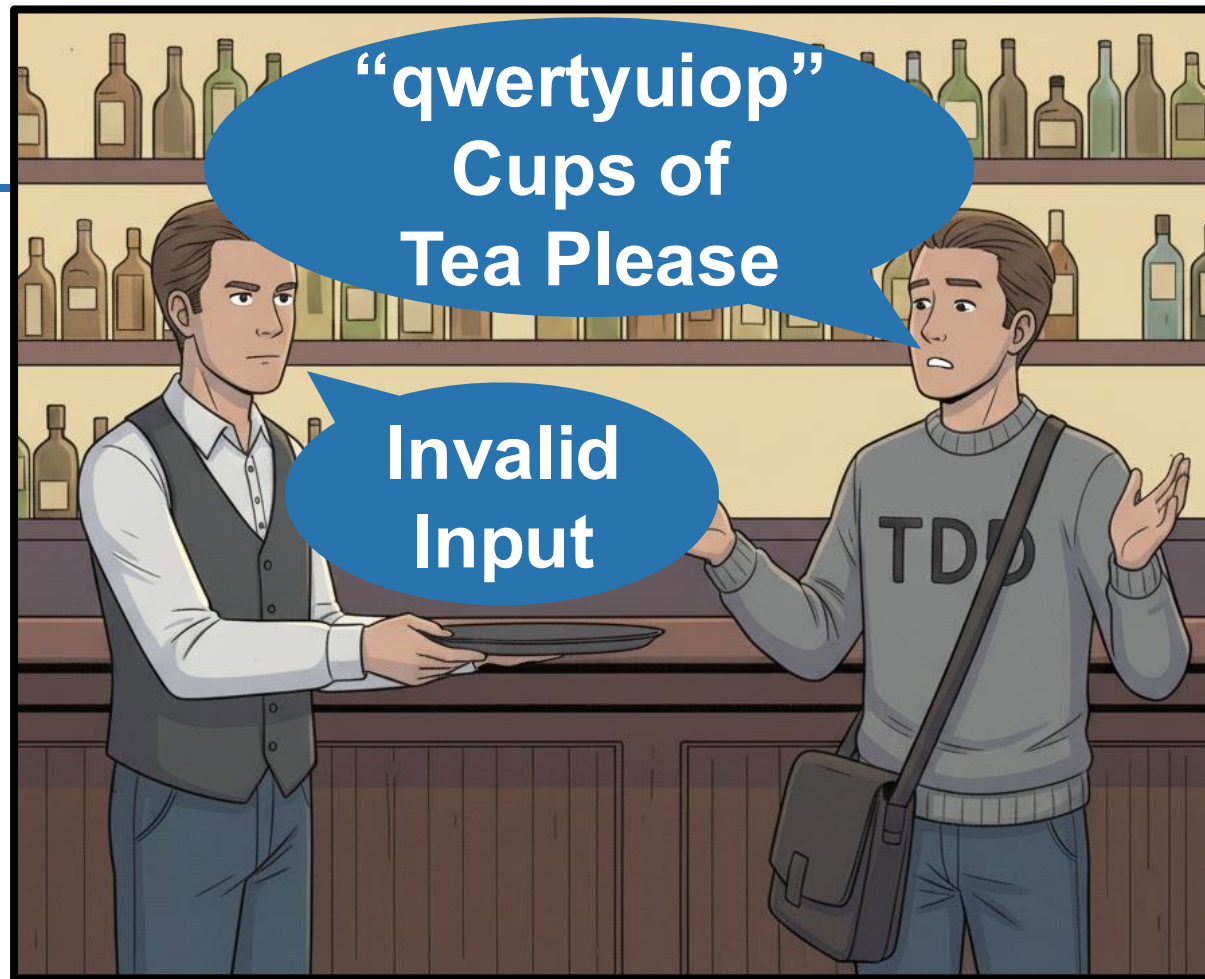
A Software Tester Orders ...

The System
Correctly
Handles an
Invalid
Request



A Software Tester Orders ...

The System
Correctly
Handles an
Invalid
Request



A Software Tester Orders ...

The System
Correctly
Handles
Errors



Let's Ship The Product!

Testing Complete!



A Real Customer Walks into the Bar

System Deployed In the Real World



A Real Customer Walks into the Bar

The Developers
And Testers
Didn't Expect
This Request



Tutorials

- <https://tobiasduerschmid.github.io/SEBook/tools/testing-foundations-tutorial>
- <https://tobiasduerschmid.github.io/SEBook/tools/tdd-tutorial>
- Test doubles: Soon-to-come

Please fill out your Exit Tickets on Bruin Learn!

Question 1

1 pts

In your own words, please summarize **three key insights** you learned about **Testing, TDD, or Refactoring** today. (Do not copy phrases from the lecture slides)

Question 2

1 pts

Besides the examples from the lecture, please describe one other concrete **scenario in which applying TDD is a good idea** and one other concrete scenario in which **applying TDD is a bad idea**. (Do not use examples from the lecture)

Question 3

Please leave any questions that you have about today's material and things that are still unclear or confusing to you (if none, simply write N/A)

Credits: These slides use images from Flaticon.com (Creators: Freepik)